Proprietary + Confidential



The extract, load, and transform data pipeline pattern

Proprietary + Confidential

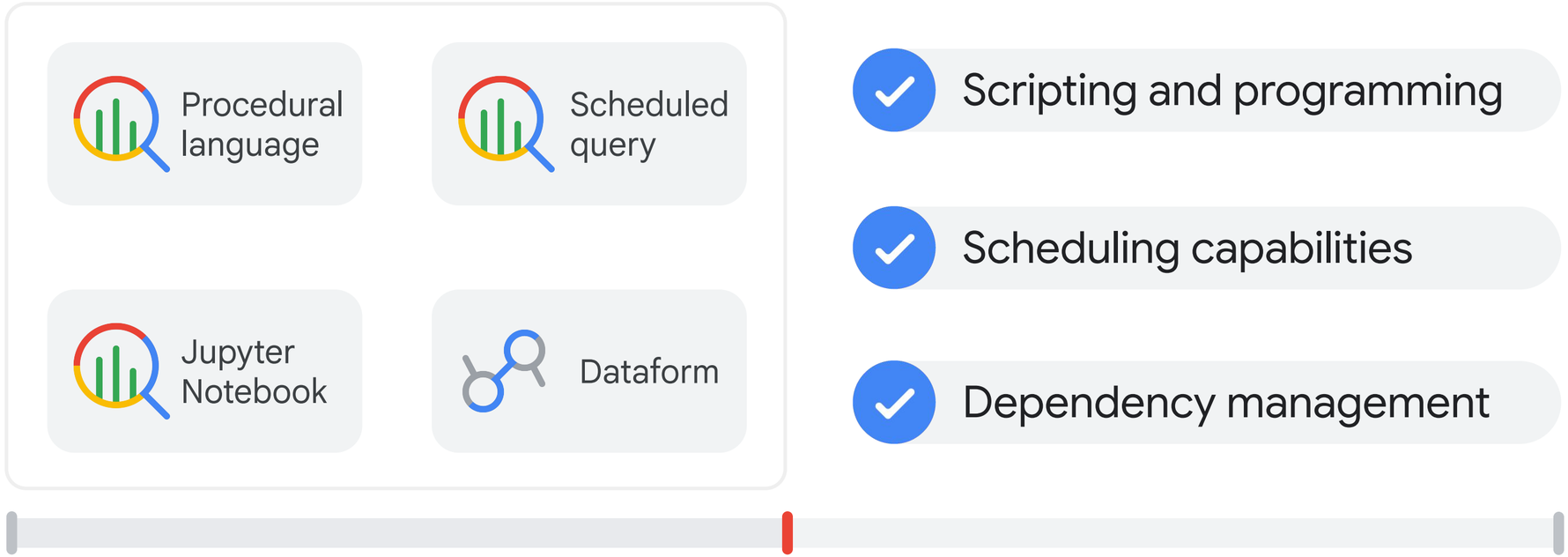
The extract, load, and transform data pipeline pattern

01	Extract, load, and transform (ELT) architecture
02	SQL scripting and scheduling with BigQuery
03	Dataform



Proprietary + Confidential

Once data is loaded into BigQuery, there are multiple ways to transform it



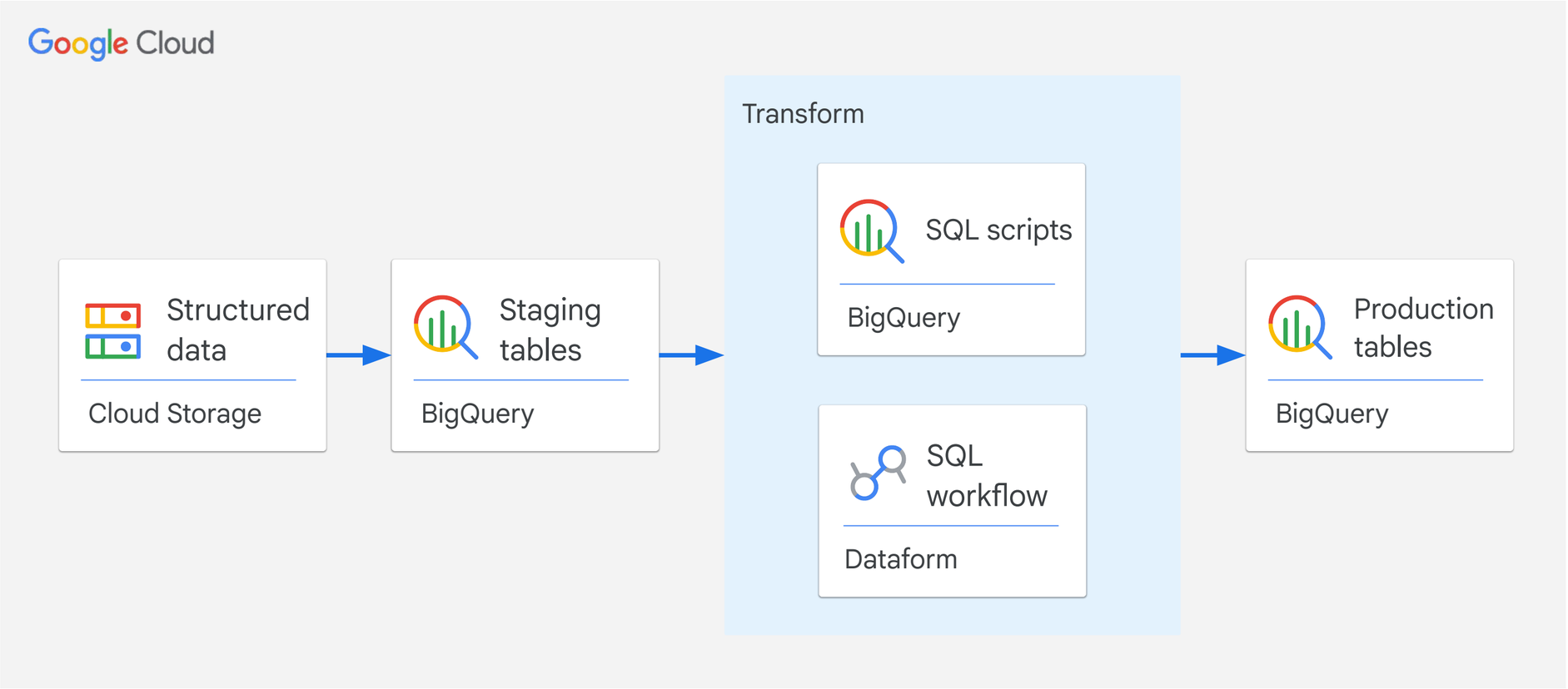
Extract and load

Extract, load, transform

Extract, transform, load

Proprietary + Confidential

Use an ELT pipeline to apply transformations after data is staged in BigQuery



Proprietary + Confidential

The extract, load, and transform data pipeline pattern

01	Extract, load, and transform (ELT) architecture
02	SQL scripting and scheduling with BigQuery
03	Dataform



Proprietary + Confidential

BigQuery supports procedural language queries to perform multiple statements in a sequence

```
-- create variables
DECLARE book_name STRING DEFAULT 'Ulysses';
DECLARE book_year INT64 DEFAULT 1922;

-- Create a temporary table called Books.
EXECUTE IMMEDIATE
  "CREATE TEMP TABLE Books
     (title STRING, publish_date INT64)";

-- add a row for Ulysses, using the variables
-- declared and the ? placeholder
EXECUTE IMMEDIATE
  "INSERT INTO Books (title, publish_date) VALUES(?, ?)"
  USING book_name, book_year;
```

Procedural language in BigQuery:

- Runs multiple statements in a sequence with shared state.
- Automates management tasks, such as creating or dropping tables.
- Implements complex logic using programming constructs like `IF` and `WHILE`.
- Can declare user-created variables or reference BigQuery's system variables.
- Can use transactions (`COMMIT`, `ROLLBACK`).

Proprietary + Confidential

User-defined functions (UDFs) transform data by using SQL or JavaScript

```
# create a permanent user-defined function
CREATE FUNCTION dataset_name.AddFourAndDivide
  (x INT64, y INT64) RETURNS FLOAT64 AS (
    (x + 4) / y
  );
```

```
# use the function in your SQL script
SELECT
  val, dataset_name.AddFourAndDivide(val, 2)
FROM ...;
```

Usage notes:

- Can be persistent (`CREATE FUNCTION`) or temporary (`CREATE TEMPORARY FUNCTION`).
- Use SQL where possible. Use JavaScript if the same cannot be accomplished with SQL.
- JavaScript UDFs can use additional input libraries.
- The `bigquery-utils` project and the `bigquery-public-data.persistent_udfs` public dataset contain community-contributed UDFs.

Proprietary + Confidential

Use Jupyter notebooks for data exploration and transformation

```
# Use BigQuery DataFrames for results not fitting into runtime memory.

import bigframes.pandas as bf
bf.options.bigquery.location = "your-location"
bf.options.bigquery.project = "your-project-id"
df = bf.read_gbq("bigquery-public-data.ml_datasets.penguins")

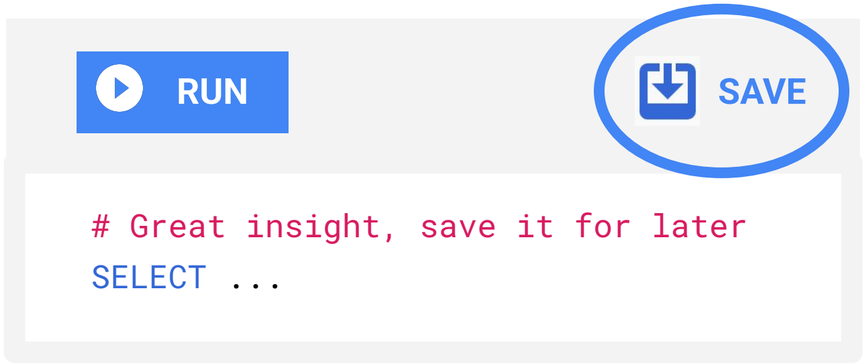
# Find the heaviest penguin species using the groupby operation to
# calculate the mean body_mass_g:
(
    df["body_mass_g"]
    .groupby(by=df["species"])
    .mean()
    .sort_values(ascending=False)
    .head(10)
)
```

Python notebooks features:

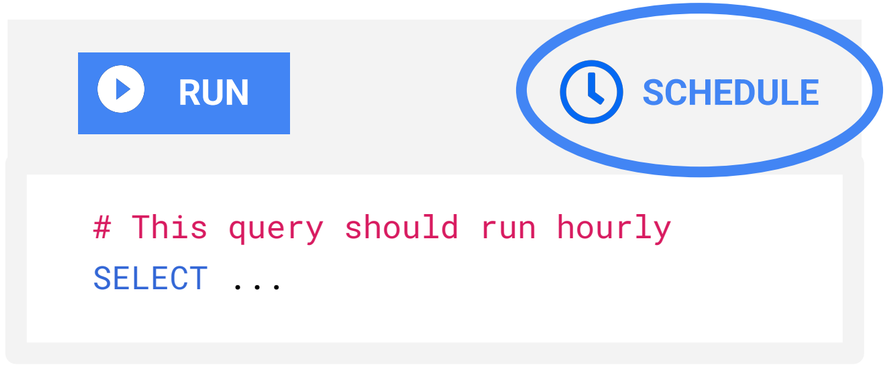
- Join datasets or aggregate data.
- Parse data from complex structures using SQL or Python functions.
- Schedule notebooks to execute at a specified frequency.
- BigQuery DataFrames is integrated into notebooks, no setup required.
- Use `matplotlib`, `seaborn`, and other popular libraries to visualize data.

Proprietary + Confidential

Save your queries and schedule them for repeating use cases



- Is visible in Explorer > Queries.
- Supports version control.
- Share with users or groups.
- Download as `.sql` file.
- Upload other queries from `.sql` files.



- Set repeat frequency.
- Set start and end times.
- Specify destination for query results.
- Can use runtime parameters.
- **Tip:** use Dataform for SQL workflows.

Proprietary + Confidential

What if you wanted to perform additional operations after query completion?



Proprietary + Confidential

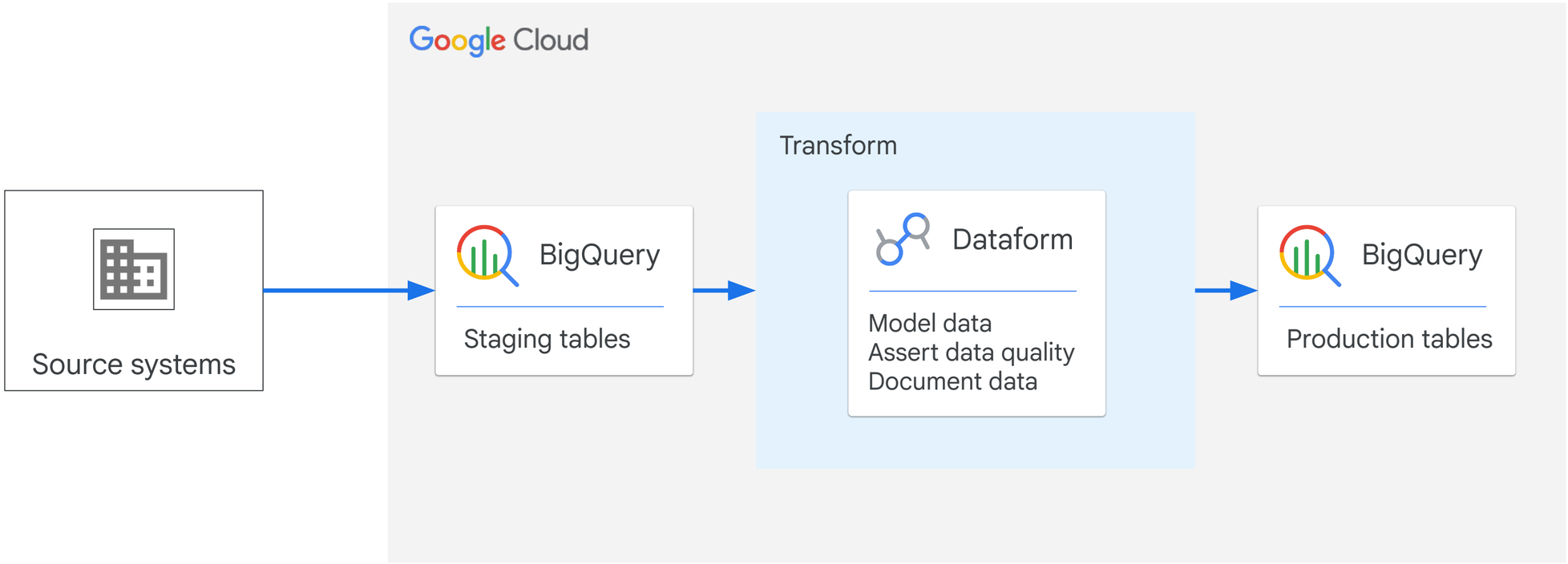
The extract, load, and transform data pipeline pattern

01	Extract, load, and transform (ELT) architecture
02	SQL scripting and scheduling with BigQuery
03	Dataform



Proprietary + Confidential

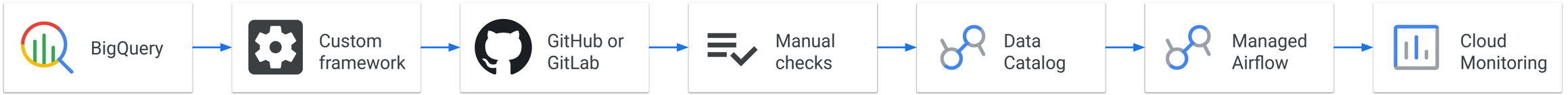
Dataform is a serverless framework to develop and operationalize ELT pipelines in SQL



Proprietary + Confidential

Dataform unifies data transformation, assertion, and automation in BigQuery

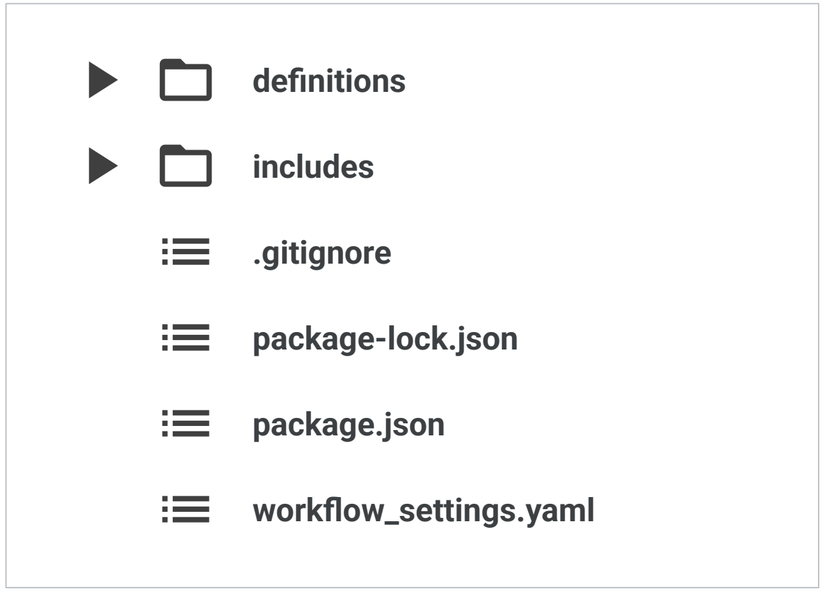
Without Dataform, this would be a time-consuming and error-prone process



Define tables in SQL	Configure dependencies	Manage code	Test data quality	Document data	Schedule pipelines	Monitor jobs

Proprietary + Confidential

Development happens in workspaces with default files and folders



You can add custom files like `README.md` as well.

- **`definitions` folder:**
  contains `sqlx` files.
- **`includes` folder:**
  contains JavaScript files.
- **`.gitignore` file:**
  lists files and directories to ignore during Git commits.
- **`package.json` file (optional):**
  defines JavaScript packages and required versions.
- **`package-lock.json` file (optional):**
  contains installed packages with exact versions.
- **`workflow_settings.yaml` file:**
  contains basic settings required to compile the project.

Proprietary + Confidential

`sqlx` file structure

- **`config` block:**

  define output table or view, dependencies, assertions and documentation.

- **`js` block:**

  define reusable constants and functions for the local SQL body.

- **`pre_operations` block:**

  define SQL to be executed before SQL body execution (e.g. creating a UDF).

- **SQL body:**

  define SQL execution and dependencies.

- **`post_operations` block:**

  define SQL to be executed after SQL body execution (e.g., granting access using IAM).

```
config {
   // specify query metadata
   // document data
   // define data quality tests
}

js {
   // define local JavaScript
   // functions and constants
}

pre_operations {
   // define SQL statements to be
   // executed before table creation
}

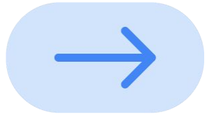
-- SQL body for core work
-- generate SQL code with JavaScript

post_operations {
   // define SQL statements to be
   // executed after table creation
}
```

Proprietary + Confidential

`sqlx` development takes care of boilerplate code by replacing redundant SQL with definitions

```
CREATE OR REPLACE TABLE
  `projectid.dataset.new_table` AS
SELECT
  country AS country,
  CASE
    WHEN country IN ('US', 'CA') THEN 'NA'
    WHEN country IN ('GB','FR','DE') THEN 'EU'
    WHEN country IN ('AU') THEN country
    ELSE 'Other countries'
  END
  AS country_group,
  device_type AS device_type,
  SUM(revenue) AS revenue,
  SUM(pageviews) AS pageviews
FROM
  `projectid.dataset.table_1`
GROUP BY
  1,2,3
```



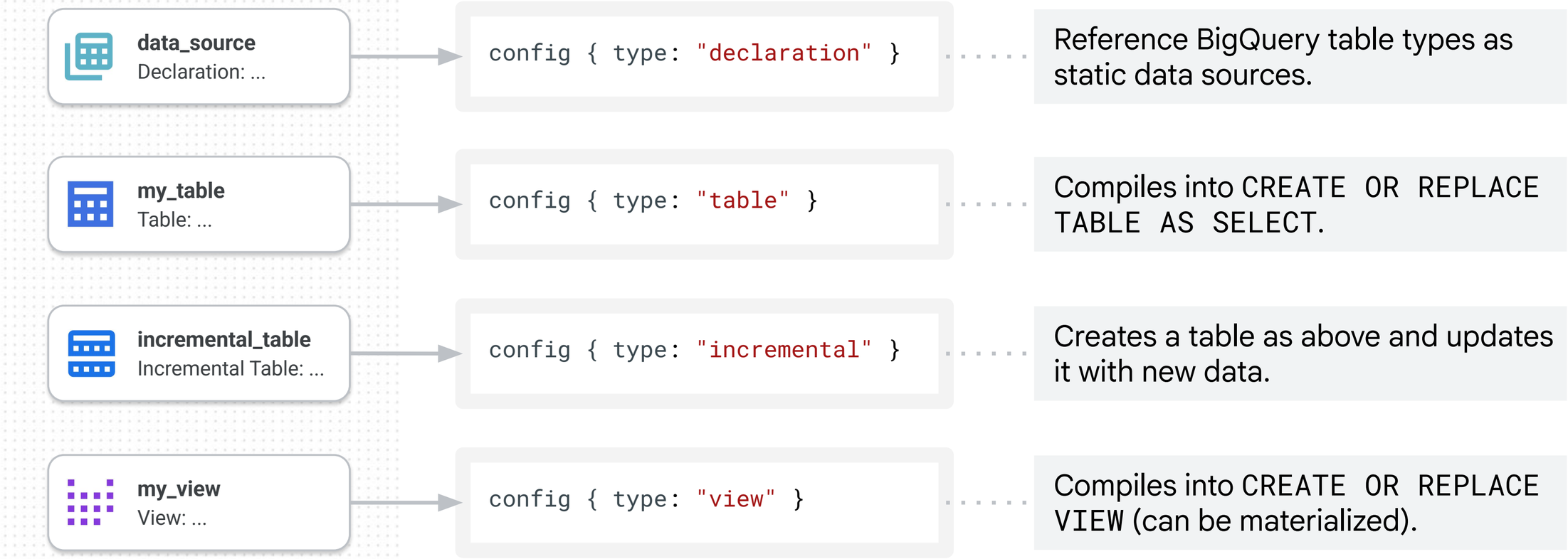
```
config { type: "table" }

SELECT
  country AS country,
  ${mapping.region("country")} AS country_group,
  device_type AS device_type,
  SUM(revenue) AS revenue,
  SUM(pageviews) AS pageviews
FROM
  ${ref("table_1")}
GROUP BY
  1,2,3
```

Google Cloud

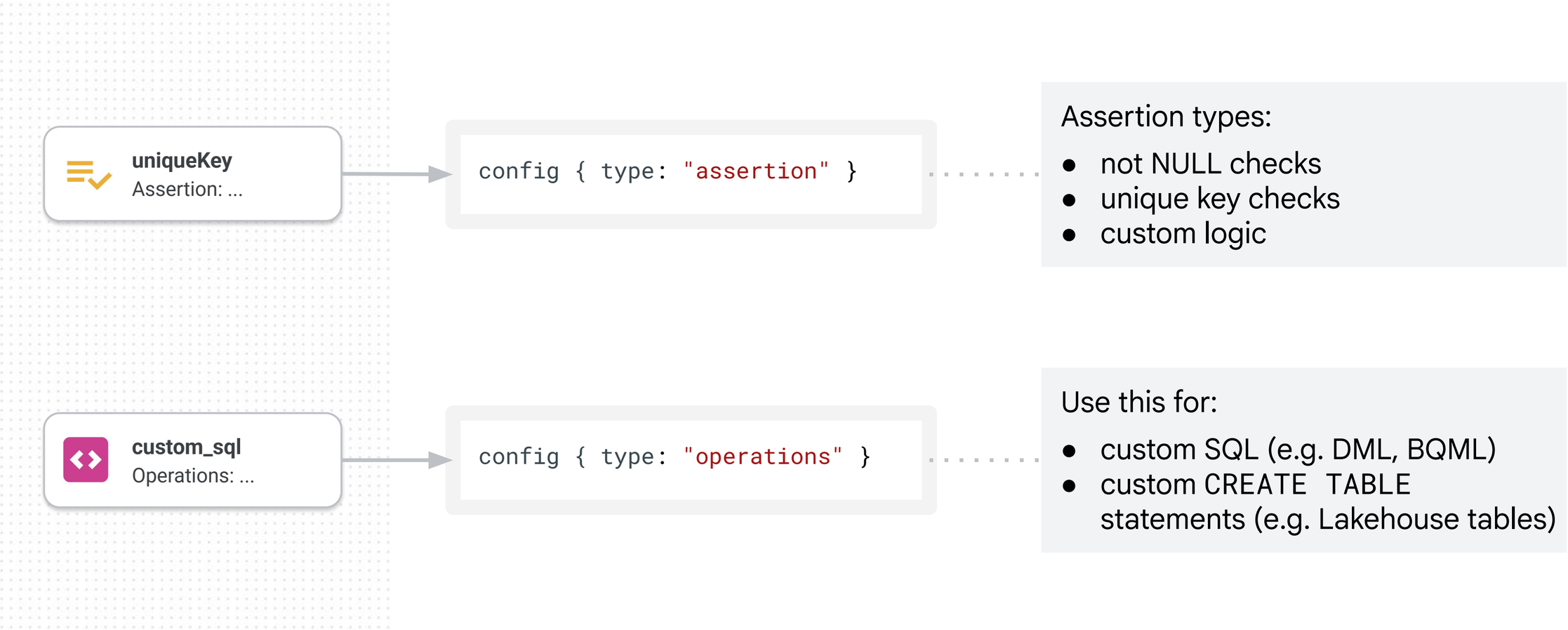
Proprietary + Confidential

Create table and view definitions that will compile into SQL statements



Proprietary + Confidential

Use assertions to define data quality tests and operations to run custom SQL statements

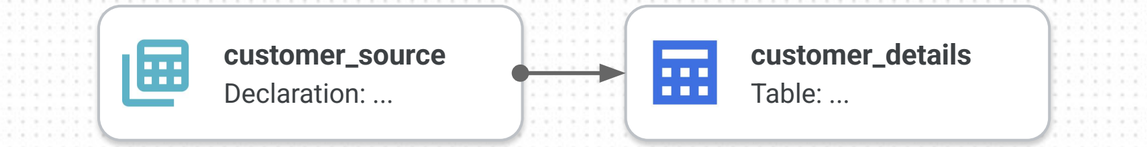


Proprietary + Confidential

Dataform compiles your definitions into SQL scripts and chains them into a workflow

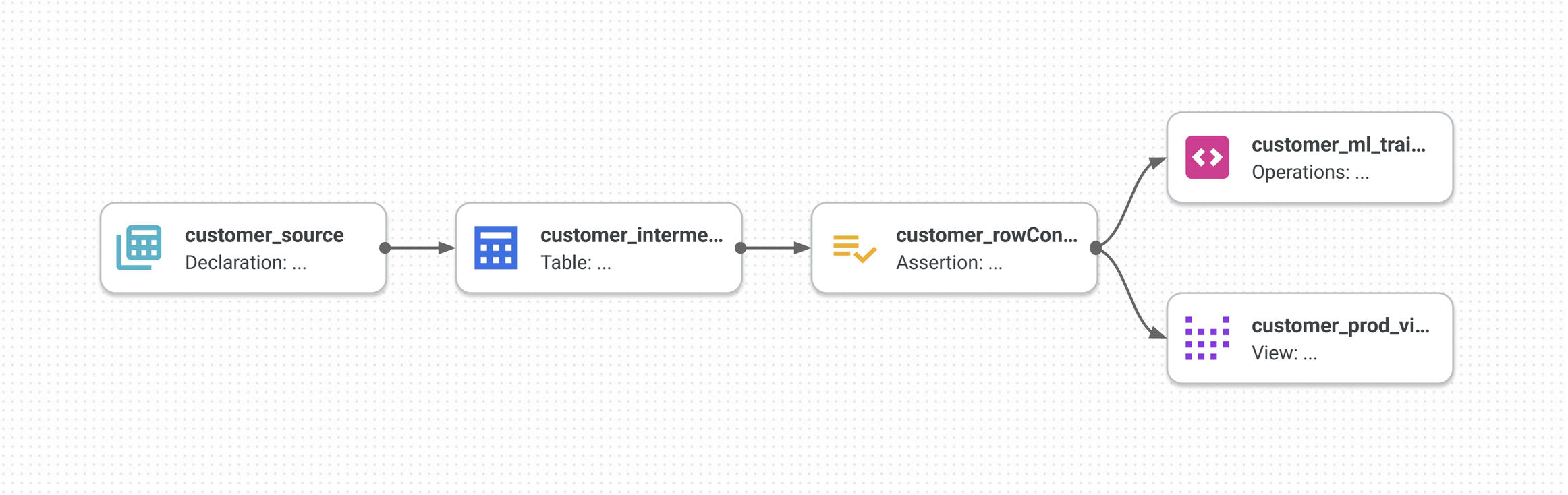
```
config {
  type: "table",
  name: "customer_details",
  description: "Customer details table",
}
SELECT ...
FROM ${ref("customer_source")}
```

```
CREATE OR REPLACE TABLE
  `<project_id>.<dataset>.customer_details`
OPTIONS(description='''Customer details table''')
AS (
  SELECT ...
  FROM `<project_id>.<dataset>.customer_source`
);
```



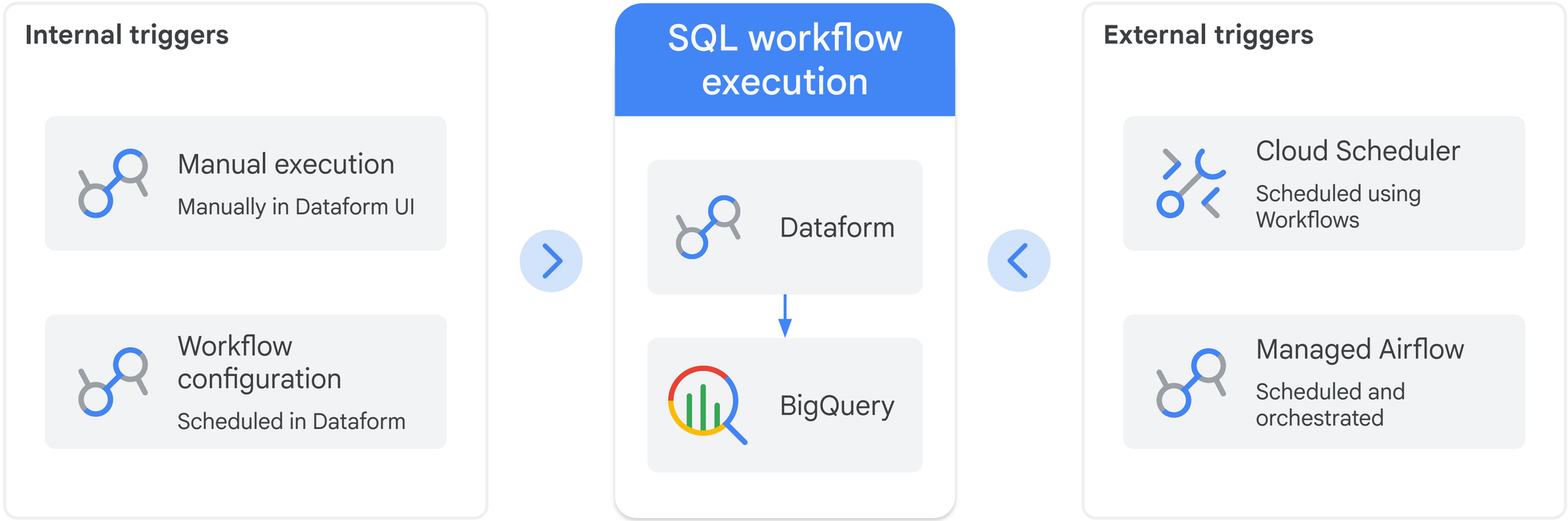
Proprietary + Confidential

The compiled graph visualizes the SQL workflow with its definitions and dependencies



Proprietary + Confidential

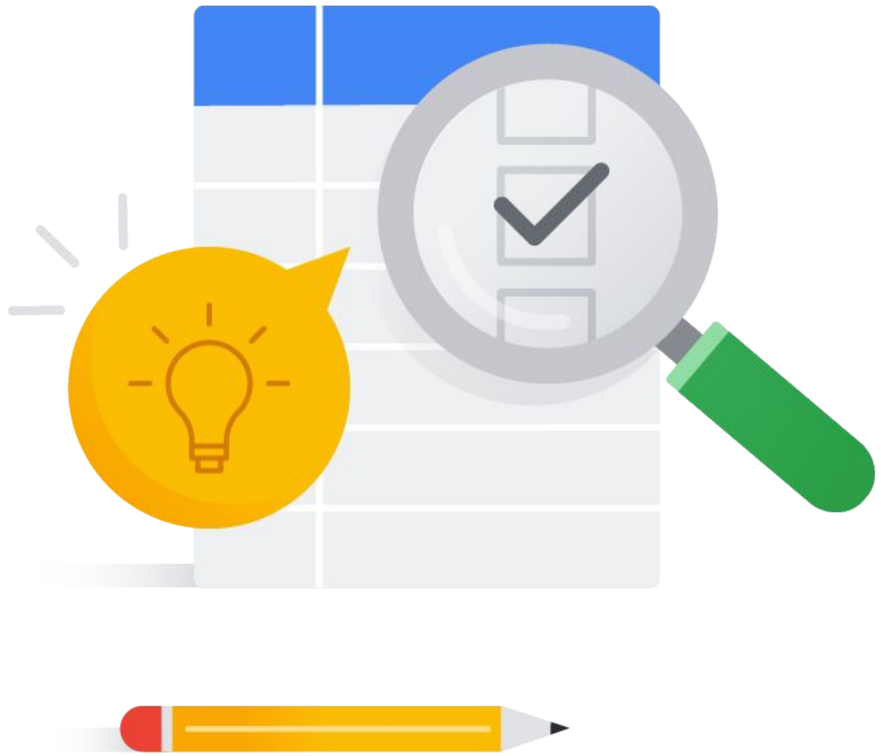
SQL workflows can be scheduled and executed on a recurring basis



Proprietary + Confidential

Lab: Create and Execute a SQL Workflow in Dataform

30 min



Learning objectives

- Create a Dataform repository.
- Create and initialize a Dataform development workspace.
- Create and execute a SQL workflow.
- View execution logs in Dataform.

Proprietary + Confidential

Lab: Create and Execute a SQL Workflow in Dataform

Data Transformation Tool



Dataform

Transformation Framework

Handle Table Dependencies

Data Warehouse



BigQuery

Proprietary + Confidential

Key Highlight: Dataform



1	**Code Versioning**: จัดเก็บสำรอง SQL อย่างมีระบบผ่าน ***Git Integration*** (GitHub / GitLab)
2	**Table Dependency Management**: ช่วยจัดลำดับการรัน Table ว่าต้องรัน Table ไหนก่อนหลัง อาทิเช่น (A > B > C) โดยอัตโนมัติ ด้วยฟังก์ชัน ***ref()***
3	**Data Quality Control**: มี ***assertions*** ช่วยตรวจสอบความถูกต้องของข้อมูล (เช่น เช็ค Null, Unique Key) ก่อนนำไปใช้
4	**Data Lineage & Governance**: มี ***Data Lineage*** คอยตรวจสอบที่มาที่ไปของ Table
5	**Incremental Logic**: มี Framework ช่วยทำ ***Incremental Load*** (อัปเดตเฉพาะข้อมูลใหม่) โดยไม่ต้องเขียน SQL MERGE ให้ซับซ้อน
6	**Orchestration Integration**: สามารถทำงานร่วมกันกับ Airflow (Cloud Compser) ได้
7	**Lakehouse Support:** รองรับ ***Lakehouse Table*** ตามที่เคยได้ลองทำใน Lab ก่อนหน้า
8	**Free of charge:** ใช้งานฟรี ไม่มีค่าใช้จ่าย เสียแค่ค่า Compute ของ BigQuery


Google Cloud